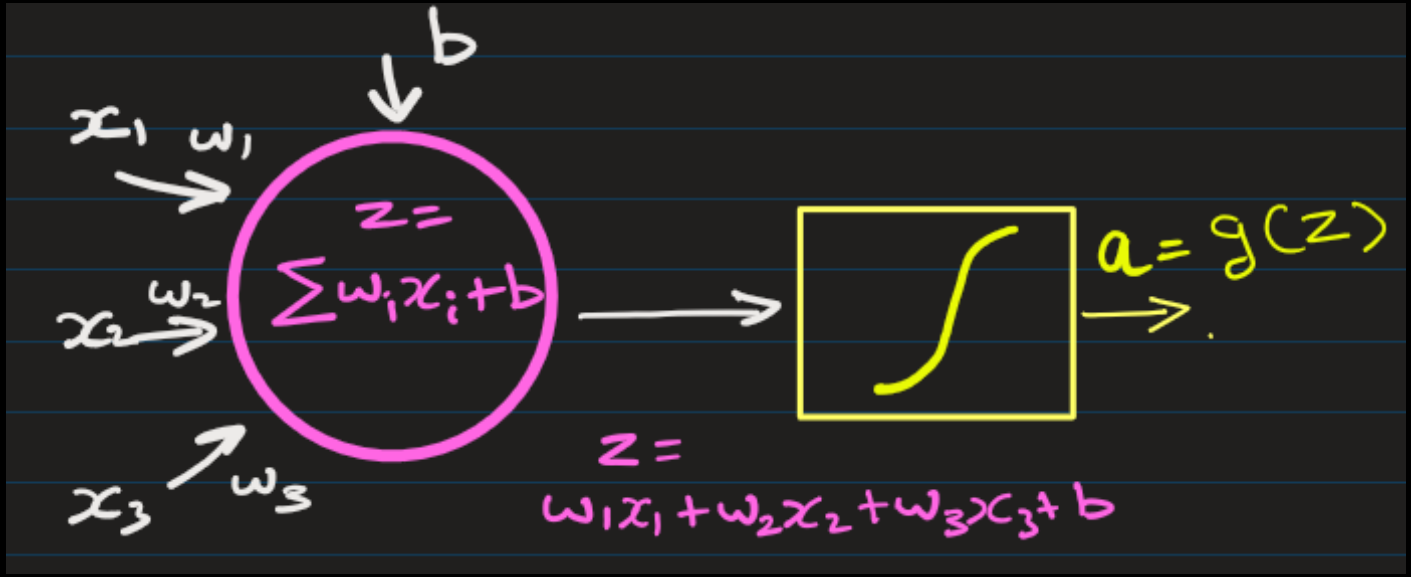
Activation Function

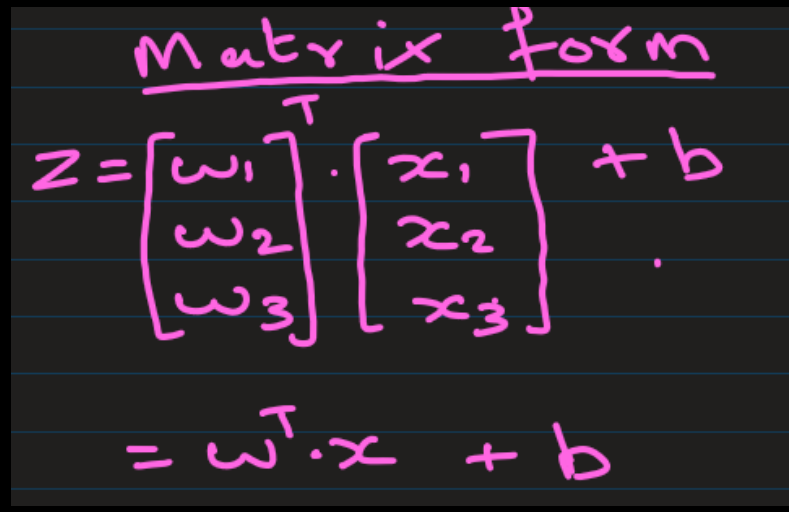
Logistic (Sigmoid)
ReLU (Rectified Linear Unit)
Hyperbolic Tangent (tanh)
Linear
SoftMax

Key Components

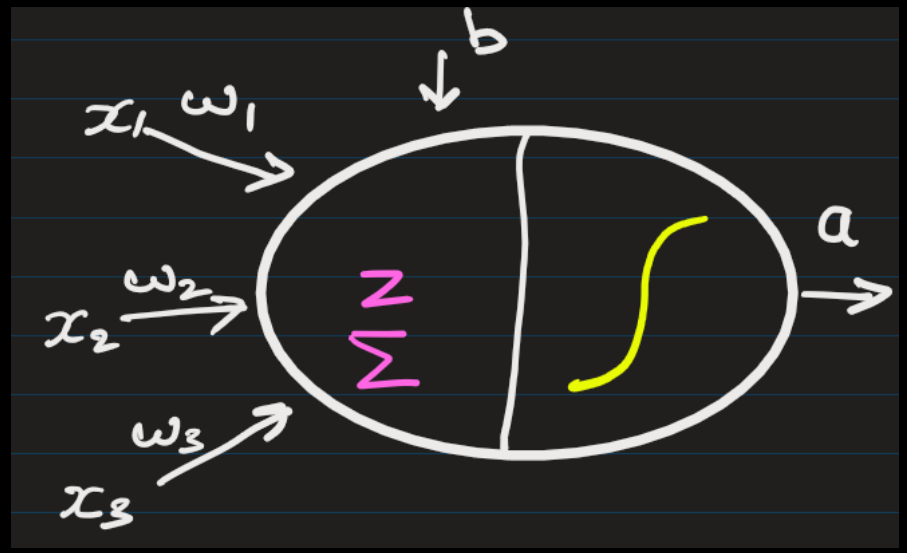
- **Neurons (Nodes)**: Each layer contains multiple neurons that perform computations.

1 Neuron ->



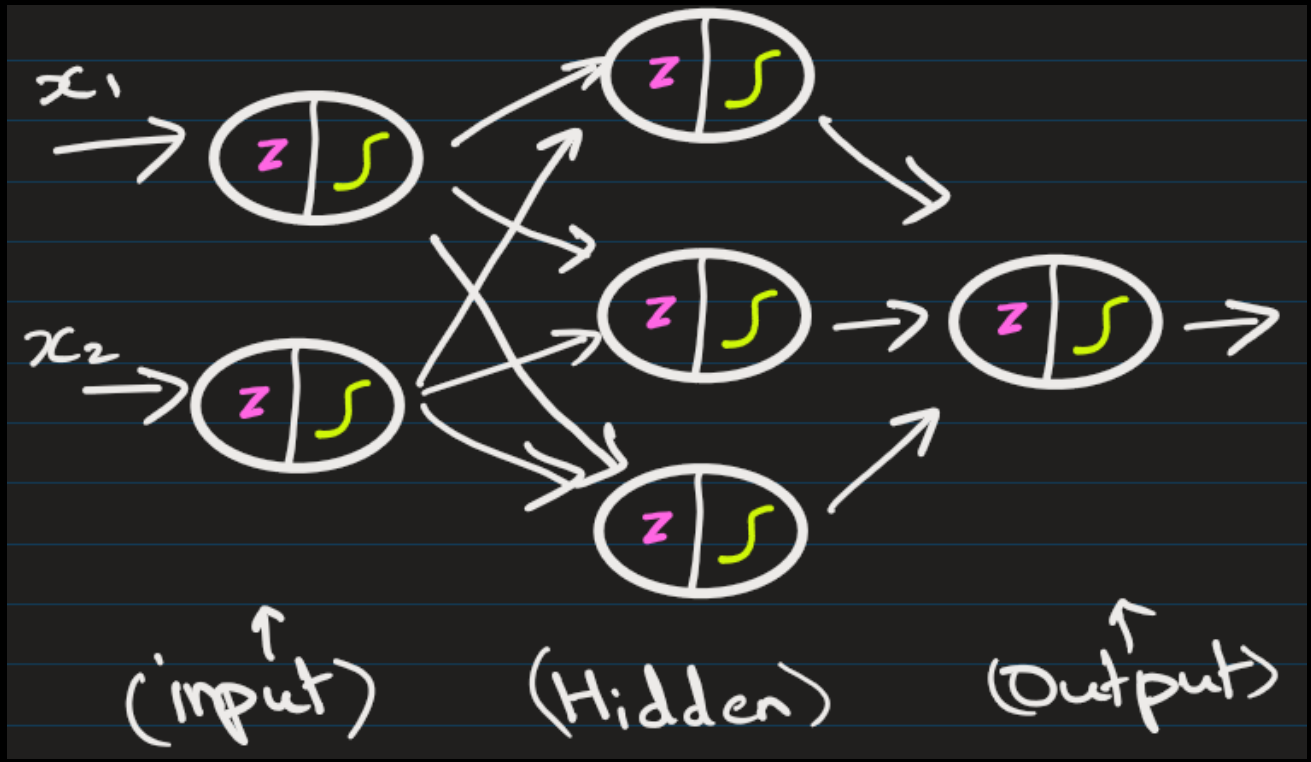


OR ->



Activation Functions

Activation functions decide whether a neuron should be activated (fire) or not

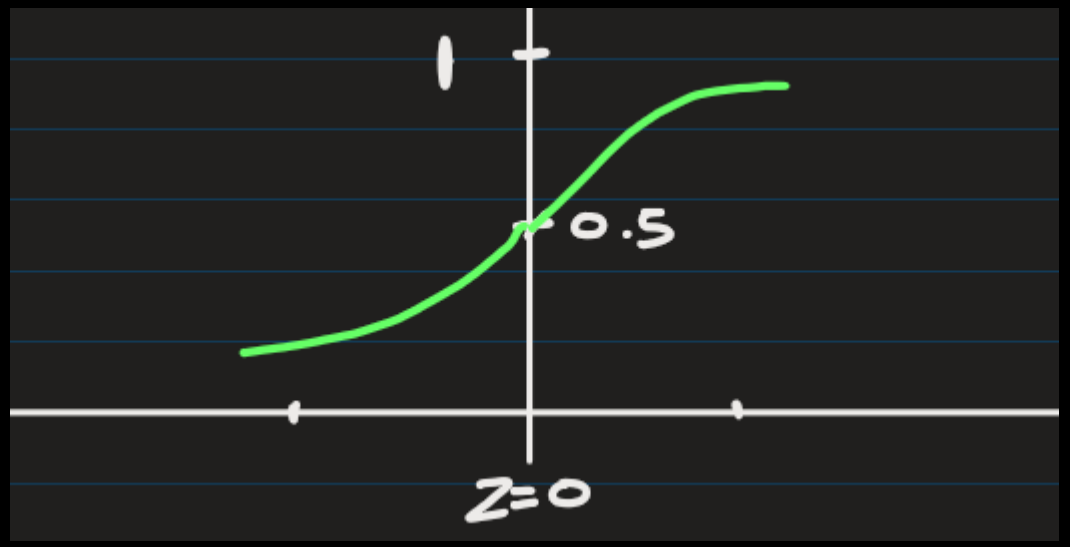


Logistic (Sigmoid) Activation Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

where:

- z = input to the activation function
- e = Euler's number (~2.718)



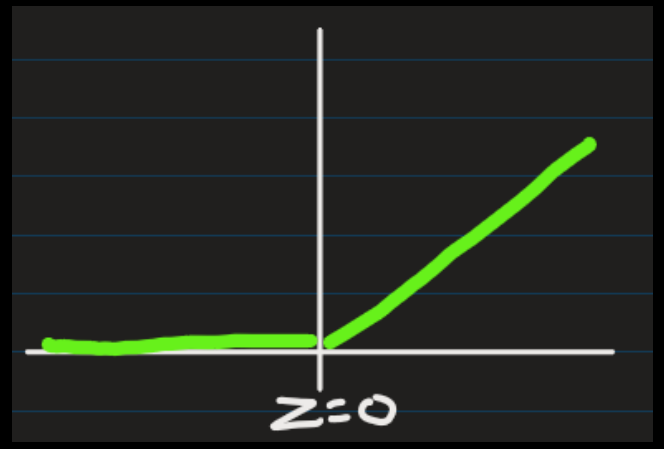
- The output is always between **0 and 1**. Because of this, it is often interpreted as a **probability**.
- Used in Binary classification output layer

ReLU (Rectified Linear Unit) Activation Function

Most widely used activation functions in modern **deep neural networks** because it is simple and computationally efficient.

$$g(z) = \max(0, z)$$

$$0 \leq g(z) < \infty$$



- The output is always between **0 and infinity**.
- Used by hidden layers, etc.

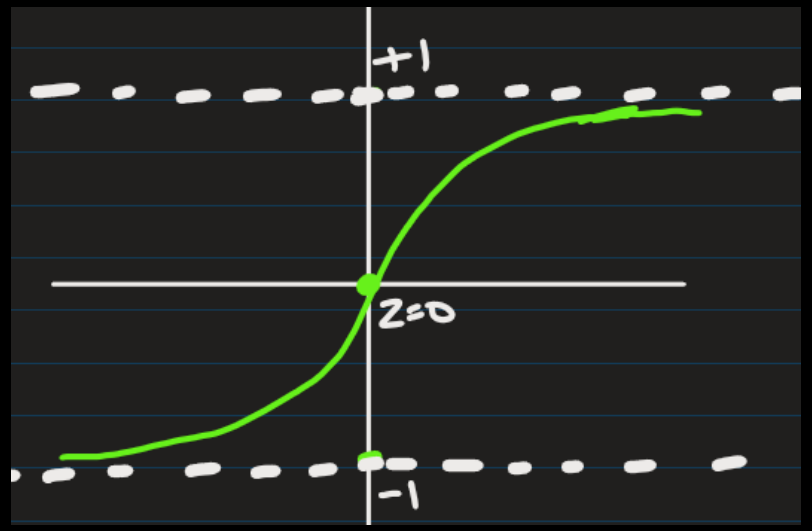
Hyperbolic Tangent (tanh) Activation Function

It is similar to the logistic (sigmoid) function but is **zero-centered**, which often helps neural networks train better.

$$g(z) = \tanh(z)$$

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$-1 < g(z) < 1$$



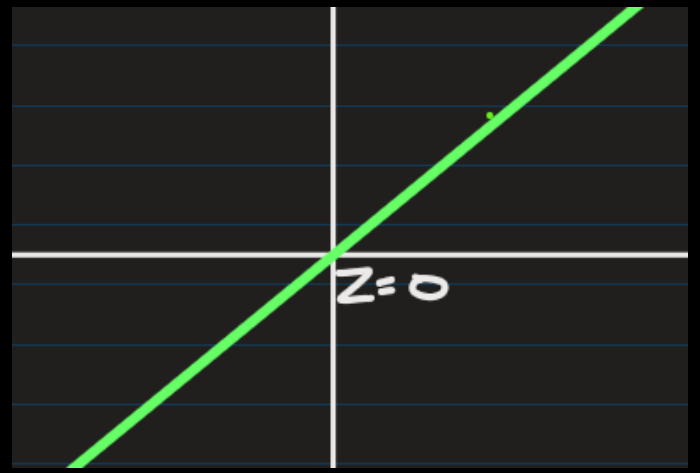
- tanh outputs **both negative and positive values**
- Used in Hidden layers in neural networks and Recurrent Neural Networks (RNNs)

Linear Activation Function

Also called the **identity function,** simply returns the input value without applying any transformation.
The output can take **any real value**.

$$g(z) = z$$

$$-\infty < g(z) < \infty$$



Generally used in the **output layer for regression problems**.
Examples:

- House price prediction
- Stock price prediction
- Temperature forecasting

SoftMax Activation Function

It is used when a neural network needs to **choose one class out of multiple classes**.
It converts raw outputs (logits) into **probabilities that sum to 1**.

For a vector of inputs $z = [z_1, z_2, \ldots, z_K]$:

$$g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

$$0 < g(z_i) < 1$$

where:

- z_i = input corresponding to class i
- K = total number of classes
- e = Euler's number (~2.718)

SoftMax operates on **an entire vector of inputs**, not just a single value.
Example input: z = [2, 1, 0.1]

$$g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

SoftMax converts this into **probabilities**.

$$[e^2, e^1, e^{0.1}] = [7.389,\ 2.718,\ 1.105]$$

$$7.389 + 2.718 + 1.105$$
$$= 11.212$$

$$g(z_1) = \frac{7.389}{11.212}$$
$$g(z_1) \approx 0.659$$

$$g(z_2) = \frac{2.718}{11.212}$$
$$g(z_2) \approx 0.242$$

$$g(z_3) = \frac{1.105}{11.212}$$
$$g(z_3) \approx 0.099$$

$$\text{Softmax}(z) = [0.659,\ 0.242,\ 0.099]$$

Predicted class = 1 (high probability)

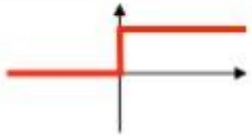
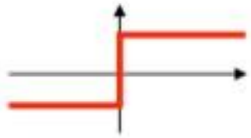
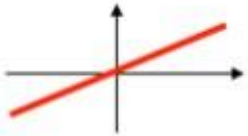
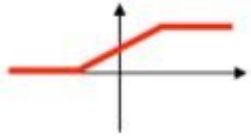
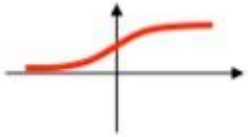
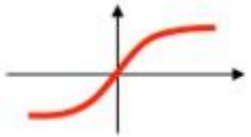
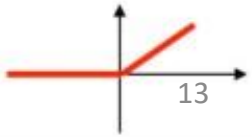
- SoftMax Probabilities Sum to 1

$$\sum_{i=1}^{K} g(z_i) = 1$$

Common Use Cases

- Output layer of multi-class classification networks
- Image classification
- Natural language processing (word prediction)
- Digit recognition

EXTRA

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = max(0, z)$	Multi-layer Neural Networks	

What type of activation unction should be used on output layer?

Choice of activation function in the output layer depends directly on the **type of problem** you're solving with an Artificial Neural Network (ANN).

1. Regression Problems

Goal: Predict continuous values (e.g., house prices, temperature).

Output activation:

- **Linear (identity)** → no activation, just raw output. f(x)=x
- Sometimes **ReLU** if the output can't be negative (e.g., predicting age, distance).

Loss function used here:
a) Mean Squared Error (MSE)
b) Mean Absolute Error (MAE) if you care more about absolute deviations.

$$L = \frac{1}{n} \sum (y_{\text{true}} - y_{\text{pred}})^2$$

Example: Predicting house price in dollars.

2. Binary Classification Problems

- Goal: Predict **two classes** (yes/no, spam/not spam).
- **Output activation:**
  - **Sigmoid** → squashes output to [0,1], interpretable as probability.

$$f(x) = \frac{1}{1 + e^{-x}}$$

Loss function used here: **Binary Cross-Entropy (Log Loss)**

Example: Predicting if a tumor is malignant (1) or benign (0).

$$L = -\frac{1}{n} \sum \left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right]$$

3. Multi-class Classification Problems (One-label)

Goal: Predict **one class out of many** (mutually exclusive).

Output activation:

- **Softmax** → gives a probability distribution across classes, all adding up to 1.

$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Loss function used here:

A. **Categorical Cross-Entropy** (when labels are one-hot encoded)

B. **Sparse Categorical Cross-Entropy** (when labels are integers, not one-hot).

$$L = -\sum y_i \log(\hat{y}_i)$$

Example: Handwritten digit recognition (0–9).

4. Multi-label Classification Problems (Multiple labels possible)
Goal: Predict **multiple independent labels** (not mutually exclusive).
Output activation:

- **Sigmoid (per output node)** → each output is treated as independent probability.

Loss function used here:
Binary Cross-Entropy applied **independently per label**.

Example: Predicting which tags apply to a movie (action, comedy, romance — could be more than one).

EXTRA

Xxxxxxxxxxx
xxxxxxxxxxxx